



Université des Sciences, des Techniques et des Technologies de Bamako

Master of Computer Science

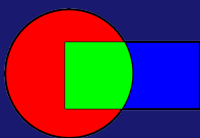
Software Engineering & Artificial Intelligence

High Performance Computing (HPC) Capstone Project

Parallel Stencil-Based Heat Diffusion Solver

HPC Capstone Project

January, 13 2026



Abstract

This document presents a comprehensive three-week capstone project for the High Performance Computing course. The project involves designing, implementing, and analyzing a parallel solver for the 2D heat diffusion equation using stencil computations. Students will implement sequential, shared-memory (OpenMP), distributed-memory (MPI), and hybrid (MPI+OpenMP) versions, conduct scalability analysis, and employ HPC debugging and profiling tools. The project aligns with all course learning objectives and provides hands-on experience with real-world HPC challenges.

Contents

1	Project Overview	1
1.1	Project Title	1
1.2	Duration	1
1.3	Team Structure	1
2	Educational Objectives	1
3	Problem Statement: 2D Heat Diffusion	1
3.1	Physical Model	1
3.2	Discretization	2
3.2.1	Spatial Discretization	2
3.2.2	Temporal Discretization	2
3.3	Finite Difference Approximation	2
3.4	Boundary Conditions	2
3.5	Initial Conditions	3
4	Algorithmic Implementation	3
4.1	Data Structures	3
4.2	Sequential Algorithm	3
4.3	Parallelization Strategy	4
5	Project Timeline and Milestones	4
5.1	Week 1: Foundation and Shared-Memory Parallelism	4
5.2	Week 2: Distributed-Memory and Hybrid Parallelism	5
5.3	Week 3: Performance Analysis and Advanced Topics	5
6	Technical Implementation Details	6
6.1	Code Structure	6
6.2	CMake Configuration	6
6.3	Key Implementation Patterns	7
6.3.1	Memory Allocation with Cache Alignment	7
6.3.2	OpenMP Parallelization with Tiling	7
6.3.3	MPI Halo Exchange	7
7	Performance Analysis Framework	8
7.1	Metrics to Collect	8
7.2	Amdahl's Law Analysis	8
7.3	Roofline Model Analysis	8
8	Debugging and Profiling	9
8.1	Debugging with GDB and MPI	9
8.2	Profiling Tools	9
8.3	Example Profiling Workflow	9
9	Experiments and Expected Results	10
9.1	Experiment 1: Strong Scaling	10
9.2	Experiment 2: Weak Scaling	10
9.3	Experiment 3: Communication vs Computation	10

10 Deliverables	10
10.1 1. Source Code Repository	10
10.2 2. Technical Report (10-15 pages)	11
10.3 3. Presentation (10-15 minutes)	11
10.4 4. Supplementary Materials	11
11 Assessment Rubric	12
12 AI Usage Policy	12
13 Resources and Support	12
13.1 Recommended References	12
13.2 Software Tools	13
13.3 HPC Cluster Access	13
14 Advanced Extensions (Optional)	13
14.1 1. GPU Acceleration	13
14.2 2. Advanced Numerical Methods	13
14.3 3. Performance Optimizations	14
14.4 4. Containerization and Reproducibility	14
Conclusion	14

1 Project Overview

1.1 Project Title

Design, Implementation, and Scalability Analysis of a Parallel Stencil-Based Heat Diffusion Solver on an HPC Cluster

1.2 Duration

Three weeks (full-time equivalent)

1.3 Team Structure

Individual work or pairs (max 2 students)

2 Educational Objectives

By completing this project, students will be able to:

1. Translate a scientific computing problem into a parallel algorithmic pattern (stencil computation)
2. Design and implement clean, modular, and efficient C/C++ code for HPC applications
3. Apply shared-memory parallelism using OpenMP with proper thread synchronization
4. Implement distributed-memory parallelism using MPI with domain decomposition and halo exchange
5. Develop hybrid parallel applications combining MPI and OpenMP
6. Deploy and execute applications on an HPC cluster using SLURM batch system
7. Conduct comprehensive performance analysis including strong and weak scalability studies
8. Apply Amdahl's law and Gustafson's law to analyze parallel efficiency
9. Debug parallel applications using HPC debugging tools (GDB with MPI support)
10. Profile applications to identify performance bottlenecks using tools like Score-P, Vampir, or gprof
11. Interpret results in the context of the roofline performance model
12. Document HPC experiments and present technical findings effectively

3 Problem Statement: 2D Heat Diffusion

3.1 Physical Model

The project simulates heat diffusion on a two-dimensional square plate using the heat equation:

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right)$$

Where:

- $T(x, y, t)$ is the temperature at position (x, y) and time t
- α is the thermal diffusivity constant (set to 1.0 for simplicity)
- The plate dimensions are $L_x \times L_y$

3.2 Discretization

3.2.1 Spatial Discretization

- Domain: $[0, L_x] \times [0, L_y]$
- Grid points: $N_x \times N_y$ cells
- Grid spacing: $\Delta x = L_x/(N_x - 1)$, $\Delta y = L_y/(N_y - 1)$
- Cell indices: $i = 0, \dots, N_x - 1$, $j = 0, \dots, N_y - 1$

3.2.2 Temporal Discretization

- Time steps: $t = 0, \Delta t, 2\Delta t, \dots, N_{\text{steps}}\Delta t$
- Use explicit forward Euler method for time integration

3.3 Finite Difference Approximation

Using central difference for spatial derivatives and forward Euler for time:

$$T_{i,j}^{n+1} = T_{i,j}^n + \frac{\alpha \Delta t}{\Delta x^2} (T_{i-1,j}^n - 2T_{i,j}^n + T_{i+1,j}^n) + \frac{\alpha \Delta t}{\Delta y^2} (T_{i,j-1}^n - 2T_{i,j}^n + T_{i,j+1}^n)$$

For a uniform grid ($\Delta x = \Delta y = h$), this simplifies to the 5-point stencil:

$$T_{i,j}^{n+1} = T_{i,j}^n + \frac{\alpha \Delta t}{h^2} (T_{i-1,j}^n + T_{i+1,j}^n + T_{i,j-1}^n + T_{i,j+1}^n - 4T_{i,j}^n)$$

For stability, the time step must satisfy:

$$\Delta t \leq \frac{h^2}{4\alpha}$$

3.4 Boundary Conditions

Implement two types of boundary conditions:

1. Dirichlet (Fixed Temperature):

$$T(0, y, t) = T_{\text{left}}, \quad T(L_x, y, t) = T_{\text{right}}, \quad T(x, 0, t) = T_{\text{bottom}}, \quad T(x, L_y, t) = T_{\text{top}}$$

2. Neumann (Insulated/Adiabatic):

$$\frac{\partial T}{\partial x}(0, y, t) = 0, \quad \frac{\partial T}{\partial x}(L_x, y, t) = 0, \quad \frac{\partial T}{\partial y}(x, 0, t) = 0, \quad \frac{\partial T}{\partial y}(x, L_y, t) = 0$$

Discretized as: $T_{0,j} = T_{1,j}$, $T_{N_x-1,j} = T_{N_x-2,j}$, etc.

3.5 Initial Conditions

$$T(x, y, 0) = T_{\text{initial}}(x, y)$$

Options include:

- Uniform temperature
- Gaussian hotspot in the center
- Multiple heat sources
- Random initial distribution

4 Algorithmic Implementation

4.1 Data Structures

```
1 // Double buffering for time stepping
2 double** T_old; // Temperature at time n
3 double** T_new; // Temperature at time n+1
4
5 // Grid parameters
6 int Nx, Ny; // Grid dimensions
7 double dx, dy; // Grid spacing
8 double dt; // Time step
9 double alpha = 1.0; // Thermal diffusivity
```

Listing 1: Core Data Structures

4.2 Sequential Algorithm

Algorithm 1 Sequential Heat Diffusion Solver

Require: $N_x, N_y, N_{\text{steps}}$, boundary_conditions, initial_condition

Ensure: Temperature field after N_{steps} time steps

- 1: Allocate T_{old} and T_{new} as $N_x \times N_y$ arrays
 - 2: Initialize T_{old} with initial condition
 - 3: Apply boundary conditions to T_{old}
 - 4: **for** $t = 1$ **to** N_{steps} **do**
 - 5: **for** $i = 1$ **to** $N_x - 2$ **do** ▷ Interior points only
 - 6: **for** $j = 1$ **to** $N_y - 2$ **do**
 - 7: $r \leftarrow \frac{\alpha \Delta t}{h^2}$
 - 8: $S \leftarrow T_{\text{old}}[i-1][j] + T_{\text{old}}[i+1][j] + T_{\text{old}}[i][j-1] + T_{\text{old}}[i][j+1] - 4T_{\text{old}}[i][j]$
 - 9: $T_{\text{new}}[i][j] \leftarrow T_{\text{old}}[i][j] + r \times S$
 - 10: **end for**
 - 11: **end for**
 - 12: Apply boundary conditions to T_{new}
 - 13: Swap($T_{\text{old}}, T_{\text{new}}$)
 - 14: **end for**
 - 15: **return** T_{old}
-

4.3 Parallelization Strategy

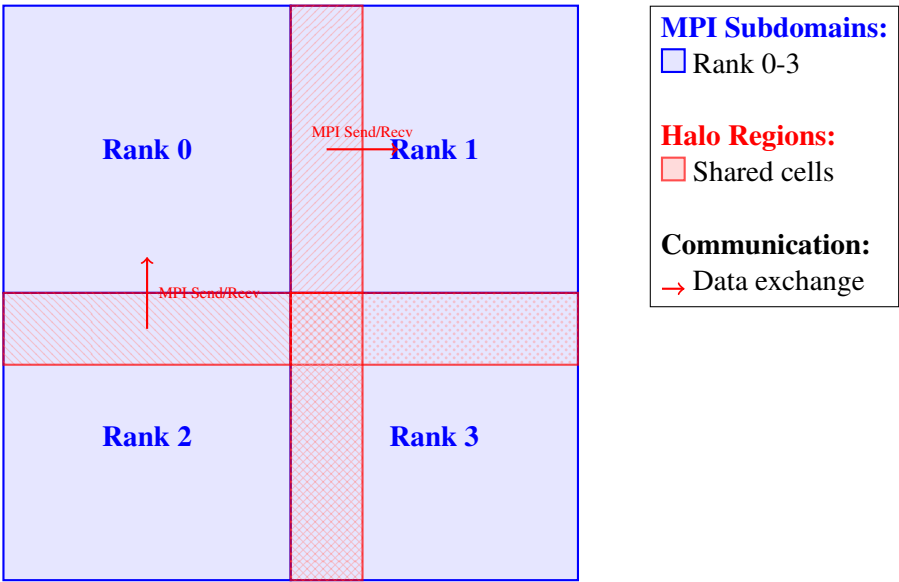


Figure 1: 2D Domain Decomposition with MPI: 4 processes with halo exchange regions

5 Project Timeline and Milestones

5.1 Week 1: Foundation and Shared-Memory Parallelism

Day	Tasks
1	- Project setup: Git repository, CMake build system - Implement sequential solver with double buffering - Add parameter parsing (grid size, time steps, boundary conditions) - Implement initial and boundary conditions
2-3	- Validate sequential solver with small test cases - Add timing instrumentation using <code>std::chrono</code> - Implement OpenMP parallelization of inner loops - Experiment with OpenMP scheduling policies (static, dynamic, guided) - Optimize memory access patterns for cache efficiency
4-5	- Strong scaling tests on a single node - Measure speedup and efficiency for OpenMP version - Analyze thread scaling limitations - Begin report writing: methodology and initial results

Table 1: Week 1 Schedule

2-3 -a)C'est une étape cruciale. Valider ton solver signifie vérifier que les mathématiques derrière le code sont justes. On ne regarde plus si le programme "tourne" sans planter, on regarde si les résultats physiques sont corrects.

5.2 Week 2: Distributed-Memory and Hybrid Parallelism

Day	Tasks
1-2	• Design MPI domain decomposition • Implement Cartesian communicator (MPI_Cart_create) • Add halo exchange using non-blocking communication (MPI_Isend/MPI_Irecv) • Handle boundary processes correctly
3-4	• Validate MPI implementation against sequential results • Implement hybrid MPI+OpenMP version • Experiment with different MPI/OpenMP ratios • Create SLURM scripts for multi-node execution
5	• Run initial multi-node tests • Compare pure MPI vs hybrid performance • Update report with distributed memory results

Table 2: Week 2 Schedule

5.3 Week 3: Performance Analysis and Advanced Topics

Day	Tasks
1-2	• Comprehensive scalability analysis: – Strong scaling: Fixed problem size (e.g., 8192×8192), vary cores 1-64 – Weak scaling: Fixed local size (e.g., 1024×1024 per process) • Measure communication vs computation ratio • Apply Amdahl's law analysis
3	• Debugging exercise: Inject and debug a parallel bug • Use GDB with MPI support • Document debugging process and findings
4	• Profiling with Score-P + Cube/Vampir • Identify performance bottlenecks • Optimize based on profiling results
5	• Final report compilation • Prepare presentation slides • Code cleanup and documentation

Table 3: Week 3 Schedule

6 Technical Implementation Details

6.1 Code Structure

```
1 hpc_project/
2 |-- CMakeLists.txt
3 |-- src/
4 |   |-- main.cpp
5 |   |-- sequential_solver.cpp
6 |   |-- openmp_solver.cpp
7 |   |-- mpi_solver.cpp
8 |   |-- hybrid_solver.cpp
9 |   |-- grid.cpp
10 |   |-- boundary.cpp
11 |   |-- io.cpp
12 |-- include/
13 |   |-- solver.h
14 |   |-- grid.h
15 |   |-- boundary.h
16 |-- scripts/
17 |   |-- run_sequential.sh
18 |   |-- run_openmp.sh
19 |   |-- run_mpi.slurm
20 |   |-- run_hybrid.slurm
21 |   |-- performance_analysis.py
22 |-- tests/
23 |   |-- test_validation.cpp
24 |   |-- test_correctness.py
25 |-- results/
26 |   |-- strong_scaling.csv
27 |   |-- weak_scaling.csv
28 |   |-- plots/
```

6.2 CMake Configuration

```
1 cmake_minimum_required(VERSION 3.10)
2 project(HeatDiffusion HPC CXX)
3
4 set(CMAKE_CXX_STANDARD 17)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6
7 # Find MPI
8 find_package(MPI REQUIRED)
9
10 # Compile options
11 add_compile_options(-Wall -Wextra -O3 -march=native -ffast-math)
12
13 # Sequential version
14 add_executable(heat_seq src/main_seq.cpp src/grid.cpp src/boundary.cpp)
15 target_compile_definitions(heat_seq PRIVATE SEQUENTIAL)
16
17 # OpenMP version
18 find_package(OpenMP REQUIRED)
19 add_executable(heat_omp src/main_omp.cpp src/grid.cpp src/boundary.cpp)
20 target_link_libraries(heat_omp OpenMP::OpenMP_CXX)
21
22 # MPI version
23 add_executable(heat_mpi src/main_mpi.cpp src/grid.cpp src/boundary.cpp)
24 target_include_directories(heat_mpi PRIVATE ${MPI_CXX_INCLUDE_PATH})
25 target_link_libraries(heat_mpi ${MPI_CXX_LIBRARIES})
26
27 # Hybrid version
```

```

28 add_executable(heat_hybrid src/main_hybrid.cpp src/grid.cpp src/boundary.cpp)
29 target_include_directories(heat_hybrid PRIVATE ${MPI_CXX_INCLUDE_PATH})
30 target_link_libraries(heat_hybrid ${MPI_CXX_LIBRARIES} OpenMP::OpenMP_CXX)

```

Listing 2: CMakeLists.txt Example

6.3 Key Implementation Patterns

6.3.1 Memory Allocation with Cache Alignment

```

1 double** allocate_grid(int nx, int ny) {
2     // Allocate contiguous memory for better cache performance
3     double* data = (double*)aligned_alloc(64, nx * ny * sizeof(double));
4     double** grid = (double**)malloc(nx * sizeof(double*));
5
6     for (int i = 0; i < nx; i++) {
7         grid[i] = &data[i * ny];
8     }
9
10    return grid;
11 }

```

Listing 3: Optimized 2D Array Allocation

6.3.2 OpenMP Parallelization with Tiling

```

1 #pragma omp parallel for collapse(2) schedule(static)
2 for (int i = 1; i < Nx-1; i++) {
3     for (int j = 1; j < Ny-1; j++) {
4         T_new[i][j] = T_old[i][j] + factor * (
5             T_old[i-1][j] + T_old[i+1][j] +
6             T_old[i][j-1] + T_old[i][j+1] -
7             4.0 * T_old[i][j]
8         );
9     }
10 }

```

Listing 4: OpenMP Implementation with Tiling

6.3.3 MPI Halo Exchange

```

1 void exchange_halos(double** local_grid, int local_nx, int local_ny,
2                     MPI_Comm cart_comm, int rank, int* coords) {
3     MPI_Request requests[8];
4     int request_count = 0;
5
6     // Exchange with left neighbor
7     if (coords[0] > 0) {
8         MPI_Isend(&local_grid[1][1], local_ny-2, MPI_DOUBLE,
9                 left_rank, 0, cart_comm, &requests[request_count++]);
10        MPI_Irecv(&local_grid[0][1], local_ny-2, MPI_DOUBLE,
11                left_rank, 1, cart_comm, &requests[request_count++]);
12    }
13
14    // Similar exchanges for right, top, bottom...
15 }

```

```

16 MPI_Waitall(request_count, requests, MPI_STATUSES_IGNORE);
17 }

```

Listing 5: Non-blocking Halo Exchange

7 Performance Analysis Framework

7.1 Metrics to Collect

Metric	Formula	Description
Execution Time	T_{total}	Wall-clock time for complete simulation
Speedup	$S(p) = \frac{T(1)}{T(p)}$	Acceleration with p processes
Parallel Efficiency	$E(p) = \frac{S(p)}{p}$	Fraction of ideal speedup achieved
Strong Scaling	Fixed N , vary p	Measure how runtime decreases with more cores
Weak Scaling	Fixed N/p , vary p	Measure how runtime changes with constant work per core
Communication Ratio	$\frac{T_{\text{comm}}}{T_{\text{total}}}$	Fraction of time spent in communication

Table 4: Performance Metrics

7.2 Amdahl's Law Analysis

$$S_{\text{Amdahl}}(p) = \frac{1}{s + \frac{1-s}{p}}$$

Where:

- s : Serial fraction (from measurements)
- p : Number of processors

Calculate s from measurements:

$$s = \frac{T_{\text{serial portion}}}{T_{\text{total}}(1)}$$

7.3 Roofline Model Analysis

Calculate operational intensity:

$$\text{Operational Intensity} = \frac{\text{Operations}}{\text{Bytes transferred}}$$

For the 5-point stencil:

- Operations per cell: 10 FLOPs (6 additions, 4 multiplications)
- Bytes transferred: 5 doubles $\times$ 8 bytes = 40 bytes (assuming no cache reuse)
- Operational intensity: $10/40 = 0.25$ FLOP/byte

8 Debugging and Profiling

8.1 Debugging with GDB and MPI

```
1 # Compile with debug symbols
2 mpicxx -g -O0 -o heat_mpi_debug src/main_mpi.cpp
3
4 # Run with debugger
5 mpirun -np 4 xterm -e gdb --args ./heat_mpi_debug
6
7 # Common GDB commands for MPI
8 (gdb) break exchange_halos
9 (gdb) info threads
10 (gdb) thread apply all backtrace
11 (gdb) p local_grid[0][0] # Check halo values
```

Listing 6: Debugging MPI Applications

8.2 Profiling Tools

Tool	Type	Use Case
Score-P + Cube	Instrumentation	Detailed profiling of MPI, OpenMP, GPU
Vampir	Trace visualization	Timeline of communication and computation
gprof	Sampling profiler	Function-level timing for sequential/OpenMP
perf	Linux perf events	Hardware counters, cache misses, instructions
mpiP	MPI profiler	MPI communication statistics
nvprof/nvvp	CUDA profiler	GPU-specific profiling (if using CUDA)

Table 5: Profiling Tools for HPC Applications

8.3 Example Profiling Workflow

```
1 # Instrument the code
2 scorep --user mpicxx -o heat_mpi_instr src/main_mpi.cpp
3
4 # Run the instrumented code
5 mpirun -np 4 ./heat_mpi_instr
6
7 # Analyze with Cube
8 cube heat_mpi_instr.cubex
9
10 # Generate trace for Vampir
11 export SCOREP_ENABLE_TRACING=true
12 mpirun -np 4 ./heat_mpi_instr
13 vampir heat_mpi_instr.otf2
```

Listing 7: Profiling with Score-P

9 Experiments and Expected Results

9.1 Experiment 1: Strong Scaling

- **Fixed problem size:** 8192×8192 grid, 100 time steps
- **Vary resources:** 1, 2, 4, 8, 16, 32, 64 cores
- **Expected observation:** Speedup increases then plateaus due to communication overhead
- **Analysis:** Fit Amdahl's law curve to determine serial fraction

9.2 Experiment 2: Weak Scaling

- **Fixed local size:** 1024×1024 per MPI process
- **Vary resources:** 1, 4, 16, 64, 256 cores
- **Total problem size grows:** 1024^2 to 16384^2
- **Expected observation:** Near-constant time per iteration for good weak scaling
- **Analysis:** Efficiency close to 1 indicates good weak scaling

9.3 Experiment 3: Communication vs Computation

- **Measure:** Time spent in halo exchange vs stencil computation
- **Vary:** Grid size and number of processes
- **Analysis:** Identify when communication becomes bottleneck
- **Expected:** For small local grids, communication dominates

10 Deliverables

10.1 1. Source Code Repository

- Git repository with complete history
- All versions: sequential, OpenMP, MPI, hybrid
- CMake build system
- Comprehensive comments and documentation
- Unit tests for validation

10.2 2. Technical Report (10-15 pages)

- Abstract and introduction
- Methodology: algorithms, parallelization strategies
- Implementation details
- Experimental setup (hardware, software, parameters)
- Results: strong/weak scaling, efficiency plots
- Analysis: Amdahl's law, roofline model, bottlenecks
- Debugging and profiling findings
- Conclusions and lessons learned
- References and appendices

10.3 3. Presentation (10-15 minutes)

- Project overview and motivation
- Key implementation challenges
- Performance results highlights
- Live demo (optional)
- Q&A session

10.4 4. Supplementary Materials

- SLURM job scripts
- Raw performance data (CSV files)
- Visualization scripts (Python/Matplotlib)
- Profiling output and traces

11 Assessment Rubric

Criterion	Weight	Description
Correctness & Validation	25%	Code produces correct results; validated against reference solutions; error analysis
Code Quality & Design	20%	Clean, modular, well-documented code; proper use of version control; efficient algorithms
Parallel Implementation	20%	Correct use of OpenMP/MPI; efficient parallel patterns; proper synchronization and communication
Performance Analysis	25%	Comprehensive scalability studies; proper use of metrics; insightful analysis of results
Debugging & Profiling	5%	Effective use of tools; identification and resolution of issues
Report & Presentation	5%	Clear, well-structured report; effective presentation of results

Table 6: Assessment Criteria

12 AI Usage Policy

- AI tools may be used for:
 - Explaining concepts and algorithms
 - Generating template code structures
 - Debugging assistance (error explanation)
 - Report writing assistance (structure, grammar)
- AI tools **may not** be used for:
 - Writing core algorithm implementations
 - Generating complete solutions
 - Avoiding understanding of key concepts
- All AI assistance must be documented in the report
- Oral defense will include detailed questions to verify understanding
- Code similarity analysis will be performed

13 Resources and Support

13.1 Recommended References

1. Gropp, W., Lusk, E., & Skjellum, A. (2014). *Using MPI: Portable Parallel Programming with the Message-Passing Interface*. MIT Press.

2. Chandra, R., Dagum, L., Kohr, D., *et al.* (2001). *Parallel Programming in OpenMP*. Morgan Kaufmann.
3. Hager, G., & Wellein, G. (2010). *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press.
4. Online: LLNL HPC Tutorials, MPI Forum, OpenMP Specification

13.2 Software Tools

- Compilers: GCC, Intel ICC, NVIDIA HPC SDK
- MPI implementations: OpenMPI, MPICH, Intel MPI
- Debuggers: GDB, TotalView, DDT
- Profilers: Score-P, Vampir, HPCToolkit, NVIDIA Nsight
- Visualization: ParaView, VisIt, Matplotlib

13.3 HPC Cluster Access

- Training cluster: [Insert your cluster details here]
- Queue limits: Max 64 cores per job, 4 hours walltime
- Storage: Home directory for code, scratch for large runs
- Support: HPC helpdesk, office hours, discussion forum

14 Advanced Extensions (Optional)

For ambitious students seeking additional challenges:

14.1 1. GPU Acceleration

- Implement CUDA or SYCL version
- Optimize for GPU memory hierarchy
- Compare CPU vs GPU performance

14.2 2. Advanced Numerical Methods

- Implement implicit methods (Jacobi, Gauss-Seidel)
- Add conjugate gradient solver for steady state
- Implement adaptive mesh refinement

14.3 3. Performance Optimizations

- Cache blocking/tiling for better locality
- SIMD vectorization (AVX-512)
- Communication avoiding algorithms
- Overlap computation and communication

14.4 4. Containerization and Reproducibility

- Create Apptainer/Singularity container
- Implement continuous integration for HPC
- Create reproducible experiment workflows

Conclusion

This comprehensive project provides students with hands-on experience in all aspects of high-performance computing, from algorithm design and parallel programming to performance analysis and optimization. By working through this project, students will develop the skills needed to tackle real-world HPC challenges in scientific computing, AI, and software engineering.

The project aligns perfectly with the course structure, covering:

- Parallel architectures (CPU, GPU, clusters)
- Core HPC algorithms (stencil computations)
- Programming models (OpenMP, MPI, hybrid)
- Performance analysis and optimization
- HPC software engineering practices
- Real-world applications

Students will emerge from this project with practical skills that are directly applicable to careers in HPC, scientific computing, and large-scale AI development.